



Згорнути меню

Оптимізація Docker-image для ML-моделей

Ми створили **повноцінний Docker-імедж для запуску ML-моделі**, використовуючи PyTorch. Зокрема, ми:

- Використали **претреновану ResNet18**, доступну в `torchvision.models`;
- Зберегли її у форматі **TorchScript** (`.pt`);
- Написали **скрипт inference**, який приймає зображення та повертає `predicted class`;
- Створили **Dockerfile** на базі `python:3.11-slim`, що містить модель, код і залежності;
- Запустили контейнер локально з реальним зображенням.

І тепер ми перейдемо до оптимізації Docker-імеджу.

! Оптимізація Docker-імеджу — це процес зменшення його розміру, підвищення безпеки, швидкості запуску та зручності використання **без шкоди для його функціональності**.

Коли ми створюємо Dockerfile для ML-моделі чи будь-якого застосунку, наша перша мета — просто «щоб воно працювало». Ми додаємо всі потрібні залежності, копіюємо код, запускаємо `pip install` — і все запускається. Це і є «робочий імедж».

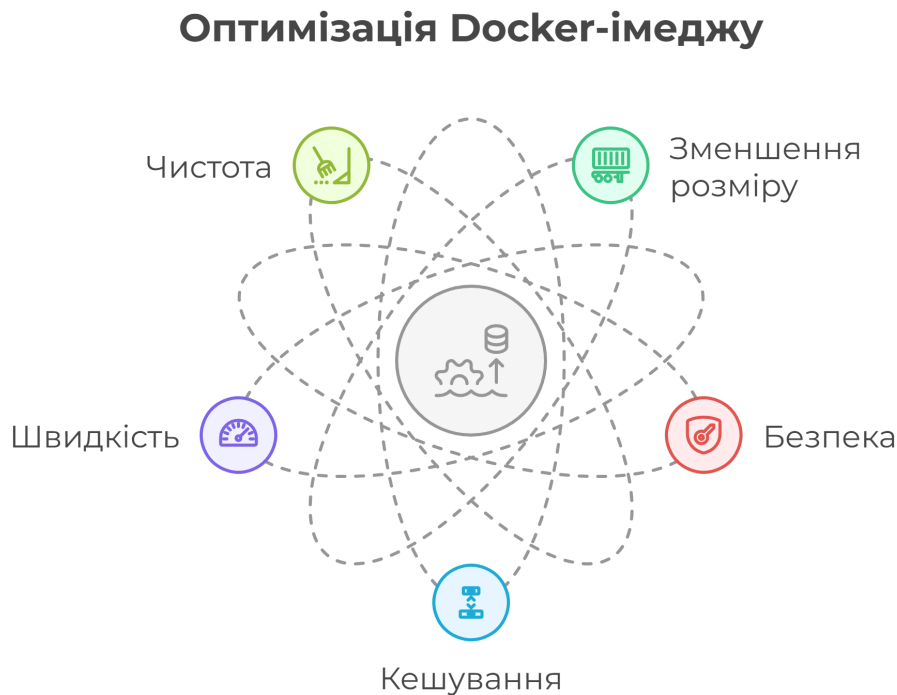
Але цей імедж **може бути набагато більшим і складнішим, ніж потрібно**. Наприклад:

- Він може містити **зайві системні пакети**, які потрібні тільки під час встановлення, а не для запуску;
- Там можуть бути **build tools**, які взагалі не потрібні під час inference;
- Залежності можуть бути встановлені в один шар без можливості кешування;
- Може використовуватись базовий образ, що важить понад 1 ГБ, хоча можна обійтися 200 МБ.

Оптимізований імедж — це такий самий імедж, який виконує ту саму задачу, але:

- Має менший розмір;
- Не містить нічого зайвого (build-залежностей, тимчасових файлів);
- Має правильну структуру шарів для кешування;
- Використовує легкі базові образи (slim , distroless , alpine);
- Іноді навіть запускається швидше через менший runtime.

Оптимізація Docker-імеджу включає кілька напрямків:



- 1. Зменшення розміру:** прибираємо все, що не потрібно для runtime, і використовуємо мінімальні базові образи.
- 2. Безпека:** зменшуємо attack surface, видаляємо build tools, shell, curl, debug tools тощо.
- 3. Кешування:** будуємо Dockerfile так, щоб шари можна було повторно використовувати в CI/CD.
- 4. Швидкість:** мінімізуємо залежності та конфігурації, які гальмують запуск.
- 5. Чистота:** структура коду та логіка імеджу відповідають best practices.

Давайте розглянемо на прикладі.

Приклад: Оптимізований Dockerfile з multi-stage

Цей варіант:

- Зменшує розмір образу;
- Розділяє «build-залежності» та «runtime»;
- Правильно кешує requirements.txt ;

- Залишає лише необхідне для запуску моделі. Dockerfile

```
# ===== STAGE 1: Install dependencies in isolation =====
FROM python:3.13-slim as builder

# Set working directory
WORKDIR /app

# Install system dependencies needed for some Python packages
RUN apt-get update && apt-get install -y --no-install-recommends \\\
    libgl2.0-0 libsm6 libxrender1 libxext6 \\\
    && apt-get clean && rm -rf /var/lib/apt/lists/*

# Copy only requirements first to leverage Docker cache
COPY app/requirements.txt .

# Install Python packages to a temporary folder
RUN pip install --no-cache-dir --target=/python-deps -r
requirements.txt

# ===== STAGE 2: Final runtime image =====
FROM python:3.13-slim

# Set working directory
WORKDIR /app

# Copy only necessary dependencies from the builder stage
COPY --from=builder /python-deps /usr/local/lib/python3.11/site-
packages/

# Copy application code and the model
COPY app/ /app/
COPY model/ /app/model/

# Default command
ENTRYPOINT ["python", "inference.py"]
```

💡 Що тут відбувається?

◆ Stage 1 — «builder»

- Ми встановлюємо system-залежності (`apt-get`) і Python-бібліотеки (`pip install`) окремо.
- Встановлення відбувається в **окремій директорії** `/python-deps` , щоб потім скопіювати.

◆ Stage 2 — «final»

- Ми беремо мінімальний `python:3.13-slim` образ **без нічого зайвого**.
- Копіюємо лише залежності зі Stage 1, код і `.pt` модель.
- Ніяких `apt`, `curl`, `build-essential` у фінальному образі немає.

Оптимізація Docker-імеджу — це не про «зробити красиво», це про **ефективність, безпеку та швидкість**.

Ми хочемо, щоб наш контейнер запускався не лише правильно, а й:

- **Швидко завантажувався** в кластер;
- **Не містив нічого зайвого** (ніяких `build-essential`, `curl`, `apt`);
- **Мав мінімум вразливостей** і легко проходив безпекові сканери;
- **Ефективно кешувався** в CI/CD або при локальній збірці.

Для цього ми застосували один із найпотужніших підходів — **multi-stage build**, який дозволяє чітко розділити етап збирання та етап виконання.

Ми також використали `python:3.13-slim` як базовий образ замість повного `python`, що дозволило значно зменшити розмір контейнера.

Тепер ви можете **зібрати новий, оптимізований Docker-образ і порівняти його з попереднім**.

Збірка нового образу:

```
docker build -t pytorch-infer-optimized .
```

Перевірити розміри обох образів:

```
docker images | grep pytorch
```

Подивитись розміри шарів усередині:

```
docker history pytorch-infer-optimized
```

Це дозволить вам наочно побачити ефект від оптимізації: менший розмір, менше шарів, чіткіша структура. У реальних проєктах така різниця в розмірі образу може зекономити **час і гроші**.

До наступного блоку

